Encapsulation

Wednesday, October 15, 2025 3:06 PM

Definition:

'''Encapsulation is a **fundamental concept of Object-Oriented Programming (OOP)** that deals with **protecting data** from direct access by users or external code.'''

*It helps in **binding data (variables)** and **methods (functions)** together into a single unit the **class** and restricts unauthorized access.

Real-World Example:
Think of a **Bank Account**.

- You can **deposit** or **withdraw** money through methods,
- But you **cannot directly access or modify** the balance stored in the system.

Encapsulation works the same way , it **hides sensitive data** and provides **controlled access** through **getter** and **setter** methods.

Access Specifiers in Python:
Access specifiers define how accessible class members (variables/methods) are.

Access Specifier	Syntax Example	Accessibility
Public	var	Accessible everywhere
Protected	_var	Accessible within class and subclasses
Private	__var	Accessible only within the class

1. Public Members:

Definition:
Public members can be accessed **anywhere** inside or outside the class.

```
class Student:
   school = "DPS"      # public class variable

   def __init__(self, name, age):
      self.name = name  # public instance variable
      self.age = age
# Accessing public members
s1 = Student("Ravi", 16)
print(s1.name, s1.age, Student.school)
```

Output:

```
Ravi 16 DPS
```

Real-life Example:

- Public members are like your **name or age** visible to others, no need to hide.

2. Protected Members
Definition:
Protected members are **meant to be used only inside the class and its subclasses**, but still can be accessed outside (by convention, not restriction).

```
class Demo:

    _a = 10
    _b = 20
    def __init__(self, c, d):
        self._c = c
```

```
        self._d = d
obj = Demo(100, 200)
print(obj._a, obj._b, obj._c, obj._d)
```

Output:

```
10 20 100 200
```

Real-life Example:

- Think of **employee performance score** — it's not secret, but only HR or managers usually access it.

3. Private Members

Definition:

Private members are declared using **two underscores __** before the name.
They **cannot be accessed directly** outside the class.

```
class Demo:
  __a = 10
  __b = 20
def __init__(self, c, d):
    self.__c = c
    self.__d = d
obj = Demo(100, 200)
# print(obj.__a) # Error
print(Demo._Demo__a)     # Accessing private class variable
print(obj._Demo__c)      #  Accessing private object variable
```

Output:

```
10
100
```

Real-life Example:

- Your **ATM PIN** or **bank account balance** completely private; can't be accessed directly, only through official methods.

1.Accessing or Modifying Private Members (Syntax):

Type	Syntax	Example
Access private **class variable**	ClassName._ClassName__variable	Company._Company__no_emp
Modify private **class variable**	ClassName._ClassName__variable = new_value	Company._Company__no_emp += 500
Access private **object variable**	object._ClassName__variable	emp1._Company__esal
Modify private **object variable**	object._ClassName__variable = new_value	emp1._Company__esal += 10000

Example: Company Class with Encapsulation

```
class Company:
  c_name = 'Qspiders'
  CEO = 'Girish'
  __no_emp = 200
  c_loc = 'Bangalore'
def __init__(self, ename, empid, design, eloc, esal, contact):
    self.ename = ename
    self.empid = empid
    self.design = design
    self.eloc = eloc
    self.__esal = esal
    self.__contact = contact
def __ch_sal(self, new):
    self.__esal += new
emp1 = Company('Priyanshu', 'QSP123', 'SE', 'Bangalore', 100000, 863492869)
```

```
# Accessing private class members
print(Company._Company__no_emp)
# Accessing private object members
print(emp1._Company__esal, emp1._Company__contact)
# Modifying values
emp1._Company__ch_sal(50000)
print(emp1._Company__esal)
```

Output:

```
200
100000 863492869
150000
```

2.Using Getter and Setter Methods:

Encapsulation becomes more meaningful when we provide **controlled access** through **getters** and **setters** instead of direct modification.

Example: Bank Class

```
class Bank:

    b_name = 'ICICI'
    b_loc = 'Kargil'
    help_line = '1800026969'
    __b_total_cash = 20000000
    def __init__(self, name, balance):
        self.name = name
        self.__balance = balance
    # Getter and Setter for object variable
    def get_balance(self):
        return self.__balance
    def set_balance(self, newbalance):
        self.__balance += newbalance
    # Getter and Setter for class variable
    @classmethod
    def get_cash(cls):
        return cls.__b_total_cash
    @classmethod
    def set_cash(cls, newcash):
        cls.__b_total_cash += newcash
ac1 = Bank('Sushanth', 100000)
print(ac1.name, ac1.get_balance())
ac1.set_balance(10000)
print(ac1.name, ac1.get_balance())
print(Bank.get_cash())
Bank.set_cash(50000)
print(Bank.get_cash())
```

Output:

```
Sushanth 100000
Sushanth 110000
20000000
20050000
```

Real-life scenario:

When a customer deposits money, they can't directly modify balance they call a method (like ATM transaction), which safely u pdates the balance internally.

3.Accessing Private Members using @property Decorator:

@property decorator allows **direct access** to private members while maintaining security.

Example:

```
class Company:

  c_name = 'Qspiders'
  CEO = 'Girish'
  __no_emp = 200
   def __init__(self, ename, empid, design, eloc, esal):
        self.ename = ename
        self.empid = empid
        self.design = design
        self.eloc = eloc
        self.__esal = esal
   # Property Decorator for getter and setter
      @property
      def esal(self):
        return self.__esal
      @esal.setter
      def esal(self, new):
        self.__esal += new
      @classmethod
      @property
      def no_emp(cls):
        return cls.__no_emp
      @no_emp.setter
      def no_emp(cls, new):
        cls.__no_emp += new
emp1 = Company('Priyanshu', 'QSP123', 'SE', 'Bangalore', 100000)
print(emp1.esal)
emp1.esal = 50000
print(emp1.esal)
print(Company.no_emp)
Company.no_emp = 100
print(Company.no_emp)
```

Output:

```
100000
150000
200
300
```

Real-life analogy:

Using @property is like **using an ATM screen**

You can check or update your balance directly but still can't touch the backend database.

Summary Points

Concept	Description	Example
Encapsulation	Binding data and methods; hiding internal details	Bank Account system
Public	Accessible everywhere	obj.name
Protected	Accessible within class and subclass	_var
Private	Only accessible within class	__var
Getter/Setter	Used to read and update private data	get_balance(), set_balance()
@property	Modern, direct way to access private data	obj.balance